

GPTAid: Automated Generation of API Parameter Security Rules via Execution Feedback and LLM Reasoning – Upgraded

Stanley Chung, UNI: syc2153

Overview

This project develops an enhanced GPTAid framework by forking the original codebase and implementing the proposed Bounded Inter-procedural Retrieval Layer and the Critic Agent. These components address the Lack of Context and the Incorrect Key Identification limitations mentioned in the original paper. The enhanced framework will be evaluated against the original one on precision, recall, bug detection rate, and average cost per API.

Research Questions

- How does bounded inter-procedural retrieval affect the precision of initial APSR generation for “wrapper” APIs?
- To what extent does a Critic Agent reduce false positives caused by incorrect key operation identification in the refinement stage?
- Which of the two proposed enhancements—the Retrieval Layer or the Critic Agent—contributes more significantly to the overall improvement in rule quality?

Value to User Community

Software Developers: Developers who integrate open source APIs can use the APSRs to ensure they are invoking APIs safely. This is especially helpful for those working in memory-unsafe languages where misuse leads to critical vulnerabilities.

Library Maintainers: The framework can be leveraged to automatically generate a solid draft of security documentation for their APIs, saving a lot of time for repetitive work.

Security Researchers & Auditors: users can leverage the framework to speed up large-scale audits, as the framework was capable of discovering vulnerabilities that were not noticed before. The automatically generated APSRs can also be a good place to start from.

Demo

The demo will be a pre-recorded video that executes the framework on one API and shows the generated APSRs.

Delivery

The project will be delivered in this public github repository:

https://github.com/aldStanley/GPTAid_expanded

Existing resources

The original framework the project expands from: <https://github.com/icy17/GPTAid/>

APIs used for evaluation:

- pcap_snapshot. Source: <https://github.com/icy17/GPTAid/>
- pcap_is_swapped. Source: <https://github.com/icy17/GPTAid/>
- pcap_major_version. Source: <https://github.com/icy17/GPTAid/>

Other APIs to be decided

Clarifications of Previous Feedbacks

Key Identification

The process of LLM comparing the right and wrong code generated in stage 2 and identifying the key that causes runtime error

Why these two specific enhancements rather than others?

It was originally considered to implement more complicated upgrades such as but not limited to refinement loops in stage 1 and stage 2, but was finalized to Retrieval Layer and Critic Agent due to budget limitation.

How did you get the idea for these two?

The idea originated from the limitations discussed in the original paper

Has anyone else extended GPTAid, which you could evaluate as a comparison baseline, or is your only experimental control GPTAid itself?

There is no other GPTAid extensions to at the time, so the original framework is the only control group

Wrt “The code base is pre-downloaded rather than dynamically retrieved online to isolate the layer and ensure our finding directly reflects the efficacy of the component.” doesn't this severely limit the codebase size you can support?

That is a valid concern, but the benefit of pre-downloading codebase outweighs the cons for this specific project for the following reasons:

- The nondeterministic nature of LLMs will introduce another layer of uncertainty/risk to the framework (requires extra resources to make sure dynamic retrieval retrieves the correct code)
- The project focuses on to what degree does mitigating the 2 limitations improves the output, so minimizing the number of variables in the experimental group makes evaluation more intuitive
- The framework is planned to be tested on only 10-20 APIs with max retrieval depth = 2 for the scope and budget of the project, meaning the codebase size to be supported is expected to be limited

In any case, do you already have a dataset of codebases? Is this a std benchmark from the literature? If so, you should also try some others. Are you targeting service APIs or library APIs or both? If your dataset is only one, should try a few samples from the other.

Only 3 API benchmarks in the original paper were accessible. Other benchmarks will be selected from the following groups, with source annotated:

- APIs calling external functions whose source code are public available
- LLM generated APIs

What does this mean? “A feedback loop is not implemented for invalid proposals due budget limitations.”

That was a typo. The correct version should be **“A feedback loop is not implemented for invalid proposals (in stage 1) due to budget limitations”**